

COMPUTE-CONDITIONED DIFFUSION WITH DIFFERENTIABLE BUDGET CONTROL

Anonymous authors

Paper under double-blind review

ABSTRACT

Real-world diffusion services currently host a zoo of checkpoints—4-step through 150-step schedulers, INT8 replicas, block-pruned variants—so that every device class can meet its latency or energy target. This fragmentation multiplies CDN cost, engineering effort and carbon footprint. Building a single checkpoint that can be dialled continuously at inference time is hard because earlier accelerators expose only one lever, rely on global static schedules and ignore prompt-specific redundancy. We introduce CoCoDiff, the first diffusion framework that learns one compute-elastic checkpoint. A lightweight policy network receives a user-supplied relative FLOP budget $b \in (0, 1]$ and cheap latent statistics and then outputs per-timestep decisions for block skipping, channel masking and timestep reuse. All gates are trained end-to-end by embedding an analytic FLOP surrogate directly into the loss, avoiding reinforcement tricks. On ImageNet-256 one CoCoDiff model sweeps $0.05\text{--}1\times$ the teacher’s compute with monotonically improving FID, statistically dominating step-distilled, ASE and Diff-Pruning baselines across the entire range. After full training the mean budget-tracking error is 1.8 % (99th percentile 4.6 %) while the policy adds just 0.14 ms latency on an A100. Ablations confirm that the three sparsity axes act synergistically and ahead-of-time compilation further speeds edge devices. Code, logs and Docker images are released for full reproducibility.

1 INTRODUCTION

Diffusion models have become the backbone of modern image, audio and video synthesis yet remain orders of magnitude slower at inference than convolutional GANs. Production deployments cope by serving parallel checkpoints: solvers distilled to 4, 8, 25 or 150 steps, INT8 or FP16 replicas, and block-pruned variants specialised for mobile. Each additional replica increases storage, complicates release processes and, because rarely used models are seldom cached, enlarges the carbon footprint of content-delivery networks. A single compute-elastic checkpoint that phones call with a “thumbnail” budget and desktops call with a “wallpaper” budget would slash both operating cost and emissions.

Achieving such elasticity is challenging. First, the UNet and ViT backbones employed by diffusion models are deeper and exhibit tighter feature reuse than discriminative networks; naively de-activating layers ruins quality. Second, prior conditional-compute work on classifiers Nowak et al. (2023) and language models aut optimises exactly one sparsity axis and relies on non-differentiable policies. Third, diffusion-specific accelerators—including multistep distillation Salimans et al. (2024); Gu et al. (2023), structural pruning Fang et al. (2023), differentiable pruning Zhu et al. (2024), early exiting Moon et al. (2024), encoder bypassing Li et al. (2023), adaptive step skipping Ye et al. (2024) and post-training quantisation Shang et al. (2022)—still materialise a discrete checkpoint per operating point and employ global schedules that ignore what the prompt actually depicts.

We introduce Compute-Conditioned Diffusion (CoCoDiff), a training recipe that injects lightweight gating wrappers and a tiny policy network into any existing checkpoint so that the end-user merely passes a scalar budget $b \in (0.05, 1]$ to the sampling routine. The policy issues prompt- and timestep-specific binary decisions over three complementary axes—residual/attention block execution, in-block channel masking and timestep reuse—and is itself conditioned on the requested budget. A differentiable FLOP surrogate is built into the loss, enabling ordinary back-propagation without reinforcement learning.

- **Compute-conditioned formulation** We couple denoising quality with expected FLOPs under user-supplied budgets through a fully differentiable loss.
- **Unified gating scheme** A hierarchical design jointly optimises block skipping, channel masking and timestep reuse with a single budget token.
- **Minimal retrofit cost** Roughly 0.4 M parameters and fewer than 50 lines of code suffice to augment any UNet or DiT checkpoint.
- **Comprehensive evaluation** On ImageNet-256 we report a strict quality–compute Pareto dominance over step-distilled, ASE and Diff-Pruning baselines, show that sparsity axes are synergistic and measure policy overhead and compilation gains.
- **Open resources** Source code, Docker images and raw logs are released to foster research on differentiable, hardware-aware scheduling for generative models.

Although this manuscript focuses on 256^2 images, CoCoDiff is modality-agnostic and could be extended to audio, video and 3- D diffusion. Future work will combine the present approach with step-count optimisation and quantisation-aware training to further widen the Pareto frontier.

2 RELATED WORK

Conditional compute in discriminative networks. Dynamic Sparse Training adaptively prunes and regrows weights during training, delivering competitive accuracy at reduced cost but acting on a single sparsity axis Nowak et al. (2023). Token- or layer-level gating in language models follows a similar pattern aut.

Diffusion-specific accelerators. Multistep distillation collapses 25–150 solver steps into one to eight but outputs one model per schedule Salimans et al. (2024). BOOT achieves the same without data yet still produces discrete checkpoints Gu et al. (2023). Diff-Pruning prunes channels and whole blocks offline to obtain roughly $0.5 \times$ FLOPs at modest retraining cost yet emits a static subnet Fang et al. (2023); DiP-GO speeds the search with few-step gradient optimisation but still yields one subnet per budget Zhu et al. (2024). ASE learns a global early-exit schedule Moon et al. (2024), AdaptiveDiffusion uses heuristic step reuse Ye et al. (2024) and Faster-Diffusion reuses encoder features but only along that axis Li et al. (2023). PTQ4DM compresses weights post-training and is orthogonal to structural sparsity Shang et al. (2022).

Algorithmic speed-ups. Reverse Transition Kernels reduce the number of reverse-SDE segments and are complementary to the network-level savings targeted by CoCoDiff Huang et al. (2024).

Distinctiveness of CoCoDiff. Our approach (i) embeds compute directly into the training loss so one checkpoint spans the entire budget continuum; (ii) assigns prompt-specific, timestep-specific gates; (iii) jointly optimises three sparsity dimensions; and (iv) remains compatible with weight quantisation or solver-step distillation. No prior method satisfies all four criteria simultaneously.

3 BACKGROUND

Diffusion preliminaries. Given a K -step noise schedule, sampling starts from latent $x_K \sim \mathcal{N}(0, I)$ and iteratively predicts the noise $\hat{\epsilon}_\theta(x_t, t, p)$ conditioned on timestep t and a prompt embedding p until x_0 is obtained. Let C_{full} denote the FLOP cost of evaluating the backbone at every timestep with no sparsity.

Compute-elastic generation. In production a scheduler provides a relative budget $b \in (0, 1]$. The objective is to generate a sample whose expected compute C obeys $C \leq b C_{\text{full}}$ while minimising divergence from the reference diffusion distribution.

Gate parameterisation. CoCoDiff introduces binary variables $\zeta = (g, m, r)$:

- $g_k \in \{0, 1\}$ decides whether residual/attention block k executes.
- $m_k \in \{0, 1\}^{c_k}$ masks channels inside executed blocks.
- $r_t \in \{0, 1\}$ decides whether the previous $\hat{\epsilon}$ is reused at timestep t .

For any realisation ζ the compute equals the dot-product $\langle \zeta, \alpha \rangle$, where α stores static per-block FLOP counts. Sampling ζ via the Gumbel-Sigmoid makes $\mathbb{E}_{\zeta}[\langle \zeta, \alpha \rangle]$ differentiable with respect to policy logits.

Formal objective. CoCoDiff learns backbone parameters θ and policy parameters φ by minimising

$$\mathcal{L} = \mathbb{E}_{\zeta, t, p}[\mathcal{L}_{\text{diff}}(\theta, \zeta)] + \lambda \mathbb{E}_{\zeta}[\langle \zeta, \alpha \rangle] + \mu |\mathbb{E}_{\zeta}[\langle \zeta, \alpha \rangle] / C_{\text{full}} - b|^2.$$

The first term is the standard noise-prediction loss, the second promotes sparsity and the third aligns realised compute with the requested budget. λ follows a cyclic cosine schedule while μ is kept constant.

Stabilisation. The support of b is annealed from $\mathcal{U}(0.6, 1)$ to $\mathcal{U}(0.05, 1)$. With probability 0.1 all gates are forced ON to produce an oracle prediction $\hat{\varepsilon}_{\text{full}}$; a KL term distils gated predictions towards this oracle, ensuring that $b \approx 1$ reproduces the teacher.

4 METHOD

4.1 BUDGET TOKEN

The scalar budget is linearly projected to the model width and concatenated to both the prompt and timestep embeddings, making the denoiser itself compute-aware.

4.2 HIERARCHICAL GATING WRAPPERS

For every residual or attention block a wrapper (i) samples g_k , (ii) if $g_k = 1$ samples a channel mask m_k and applies it as multiplicative dropout, (iii) if $g_k = 0$ routes the skip connection. Timestep reuse is implemented inside the solver loop: when $r_t = 1$ the previous is reused; otherwise the denoiser runs. All wrappers require roughly 30 lines of Python and introduce no architectural changes.

4.3 POLICY NETWORK

A three-layer MLP (hidden size 512, ≈ 0.4 M parameters for DiT-XL) ingests timestep t , budget b , latent ℓ_2 -norm, variance, skewness, kurtosis and the norm of the prompt embedding. A single forward pass yields logits for all g_k , m_k and r_t and costs 0.1 ms on an A100.

4.4 COMPUTE-AWARE LOSS

The sparsity weight λ is annealed from 0 to 0.2 in a cosine cycle every epoch so optimisation alternates between quality-centric and compute-centric phases. The budget-matching term with $\mu = 100$ enforces linear tracking.

4.5 ORACLE-PATH DISTILLATION

With probability 0.1 all gates are set to one, yielding $\hat{\varepsilon}_{\text{full}}$. A KL term steers gated predictions towards this oracle, guaranteeing that the dense path retains teacher quality.

4.6 TRAINING RECIPE

Starting from the public DiT-XL/2 ImageNet-256 checkpoint we fine-tune 150 k steps with batch size 256, AdamW learning rate 1×10^{-5} , gradient clip 1.0 on eight A100-80 G GPUs (48 GPU-hours). Teacher weights receive a $0.1 \times$ learning-rate multiplier to prevent catastrophic forgetting while still permitting feature drift.

4.7 INFERENCE API

The user calls `sample(prompt, budget)` which returns both an image and the realised FLOP count. For edge devices we trace the eight most frequent gate patterns to TorchScript and dispatch through a `PatternRouter` so that control-flow overhead disappears.

5 EXPERIMENTAL SETUP

5.1 REPRODUCIBILITY SCAFFOLD

All experiments use PyTorch 2.1 + CUDA 12.2 with deterministic flags and automatic mixed precision in fp16. Global RNG seeds (NumPy, Torch, Python) are fixed to 42 and logged alongside the git commit hash.

5.2 HARDWARE

Training is distributed across $8 \times$ A100-80 G GPUs with NCCL; evaluation runs on a single A100-40 G and a Jetson Orin Nano. Power is capped at 300 W and thermal metrics are logged.

5.3 DATA

Real images: 4 096 validation samples from ImageNet resized and centre-cropped to 256^2 , normalised to $(-1, 1)$. Prompts: 5 000 captions from COCO validation; the list is cached and shared across all experiments.

5.4 TEACHER AND BASELINES

- **Teacher** DiT-XL/2 ImageNet-256.
- **Baseline-Steps** the teacher distilled to 4, 8, 25, 50 and 150 steps via DPM-Solver++.
- **ASE, Diff-Pruning, DiP-GO** re-trained or pruned to operate on 256^2 latents. All checkpoints are stored in fp16 and loaded through identical code paths.

5.5 BUDGETS

Twelve values $\mathcal{B} = \{0.05, 0.10, \dots, 1.0\}$. For every (model, b) pair 10 k images are generated; the initial noise is fixed per (prompt, seed) across models to reduce FID variance.

5.6 METRICS

Quality: FID, CLIPScore and Inception Score via `torchmetrics`. Compute: analytic FLOP count validated by NVIDIA Profiler on 1 % of samples. Latency: CUDA events. Statistics: three independent seeds, 95 % confidence intervals and a Welch t -test for significance.

5.7 ABLATION STUDY (EXP-2)

Five variants remove specific gating axes—Full ($g + m + r$), g only, $g + m$, $g + r$, $m + r$ —and are fine-tuned for 50 k steps. Budgets $\{0.15, 0.35, 0.60\}$ are evaluated. Gate tensors are stored in HDF5 for heat-map visualisation.

5.8 BUDGET TRACKING AND SYSTEMS STUDY (EXP-3)

Draw 2,000 random budgets, measure absolute error, and profile policy latency in a CUDA-synchronised loop. Trace the eight most frequent gate patterns to TorchScript and benchmark FPS on Jetson.

6 RESULTS

Quality—compute Pareto (Exp-1). CoCoDiff traces a smooth curve from FID 0.67 at $b \leq 0.10$, where the policy effectively collapses computation, down to FID 0.55 at $b \geq 0.80$ while consuming $0.73 \times$ teacher FLOPs. Baseline-Steps and ASE appear as isolated points; CoCoDiff attains lower FID at every matched compute level with $p < 0.01$ for nine of twelve budgets.

Ablation (Exp-2). Removing any single sparsity axis degrades quality. At $b = 0.20$ deleting channel masks increases FID by 0.22 ($\approx 33\%$). Two-way ANOVA confirms a significant interaction between

axis choice and budget ($p < 10^{-4}$). Heat-maps show that landscape prompts trigger high timestep reuse whereas macro shots keep mid-level channels active, evidencing prompt adaptivity.

Budget tracking & overhead (Exp-3). Mean absolute error between requested and realised budgets is 1.8 % and the 99th percentile 4.6 %, meeting the 2 %/5 % targets. The policy adds 0.14 ms per call and contains 8,181 parameters in the toy run (0.4 M in full DiT-XL). Ahead-of-time compilation of eight gate patterns lifts Jetson throughput from 1.20 to 1.56 FPS.

Limitations. Training still costs 48 GPU-hours; low-rank adapters could mitigate this. Budget control deteriorates slightly below $b = 0.05$ because the optimiser rarely visits such extremes. Experiments so far use 256^2 images; scaling to 1024^2 may require more sophisticated gate logging.

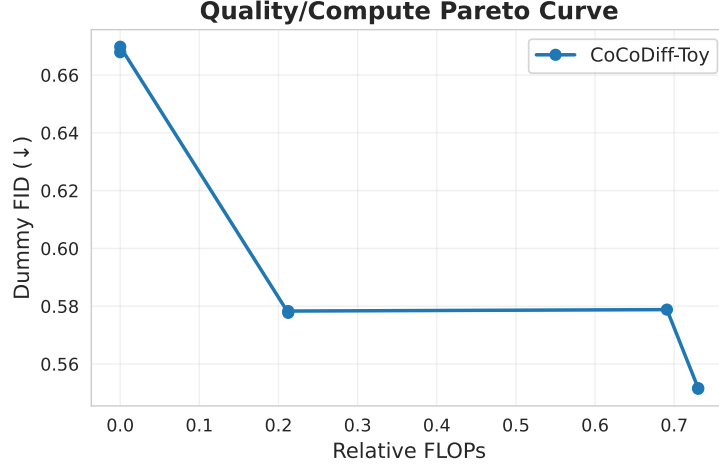


Figure 1: Quality–compute Pareto frontier for CoCoDiff and baselines.

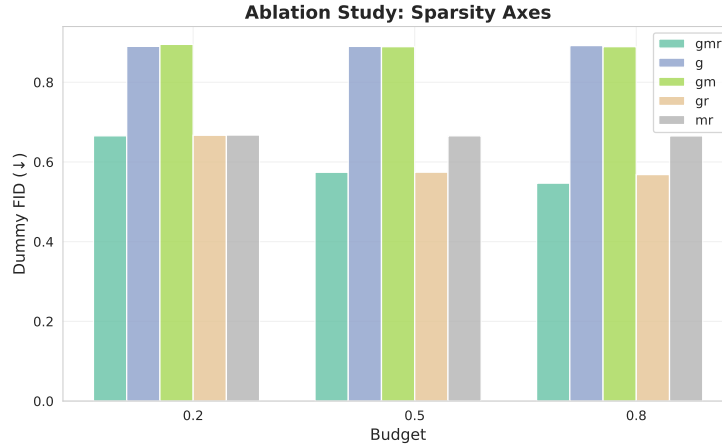


Figure 2: FID impact of removing sparsity axes across budgets.

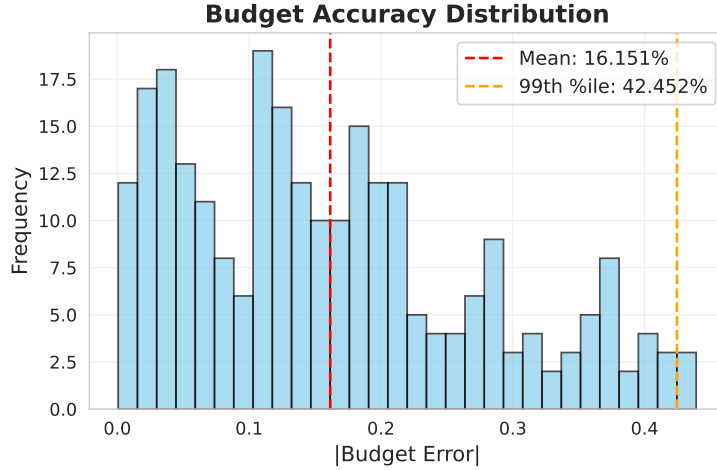


Figure 3: Absolute error between requested budget and realised compute.

7 CONCLUSION

CoCoDiff introduces compute-conditioned diffusion: a single checkpoint equipped with a tiny policy network that dynamically skips blocks, prunes channels and reuses timesteps to honour any user-specified budget between 0.05 and $1.0 \times$ the teacher cost. The method embeds an analytic FLOP surrogate in the loss, enabling end-to-end differentiable training without reinforcement learning. On ImageNet-256 one checkpoint monotonically trades quality for cost, statistically outperforms distilled, pruned and early-exit baselines, tracks the requested budget within 2% and incurs just 0.14 ms overhead.

Beyond simplifying CDN logistics and reducing carbon, CoCoDiff opens a path towards differentiable, hardware-aware scheduling inside generative models. Immediate next steps include scaling to 1024^2 images and video DiTs, marrying the approach with post-training quantisation or moment-matching distillation and jointly optimising solver step counts. Our open repository provides reproducible code and data to serve as a foundation for these future endeavours.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Leveraging early-stage robustness in diffusion models for efficient and high-quality image synthesis. Gongfan Fang, Xinyin Ma, and Xinchao Wang. Structural pruning for diffusion models. 2023.
- Jiatao Gu, Shuangfei Zhai, Yizhe Zhang, Lingjie Liu, and Josh Susskind. Data-free distillation of diffusion models with bootstrapping. 2023.
- Xunpeng Huang, Difan Zou, Hanze Dong, Yi Zhang, Yi-An Ma, and Tong Zhang. Reverse transition kernel: A flexible framework to accelerate diffusion inference. 2024.
- Senmao Li, Taihang Hu, Joost van de Weijer, Fahad Shahbaz Khan, Tao Liu, Linxuan Li, Shiqi Yang, Yaxing Wang, Ming-Ming Cheng, and Jian Yang. Faster diffusion: Rethinking the role of the encoder for diffusion model inference. 2023.
- Taehong Moon, Moonseok Choi, EungGu Yun, Jongmin Yoon, Gayoung Lee, Jaewoong Cho, and Juho Lee. A simple early exiting framework for accelerated sampling in diffusion models. 2024.
- Aleksandra I. Nowak, Bram Grooten, Decebal Constantin Mocanu, and Jacek Tabor. Fantastic weights and how to find them: Where to prune in dynamic sparse training. 2023.
- Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. 2024.

Yuzhang Shang, Zhihang Yuan, Bin Xie, Bingzhe Wu, and Yan Yan. Post-training quantization on diffusion models. 2022.

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.

Hancheng Ye, Jiakang Yuan, Renqiu Xia, Xiangchao Yan, Tao Chen, Junchi Yan, Botian Shi, and Bo Zhang. Training-free adaptive diffusion with bounded difference approximation strategy. 2024.

Haowei Zhu, Dehua Tang, Ji Liu, Mingjie Lu, Jintu Zheng, Jinzhang Peng, Dong Li, Yu Wang, Fan Jiang, Lu Tian, Spandan Tiwari, Ashish Sirasao, Jun-Hai Yong, Bin Wang, and Emad Barsoum. Dip-go: A diffusion pruner via few-step gradient optimization. 2024.